UNIVERSIDADE PAULISTA
CIÊNCIA DA COMPUTAÇÃO

APS do segundo semestre de 2026

Avaliação da qualidade do sistema Estoque Bairro em Java

Relatório técnico de apoio à entrega acadêmica

Integrantes e RA: [preencher com os dados reais do grupo]

Professor orientador: [preencher]
Turma e campus: [preencher]

Versão do sistema 1.0.0
Consolidação técnica em 10 de novembro de 2026

Este documento reúne a implementação e as evidências técnicas disponíveis. O grupo deve completar a avaliação com participantes, a identificação, as fichas individuais e a revisão de normalização antes de entregar. Não há atividades ou assinaturas atribuídas aos alunos.

Campinas
2026

Resumo

Este trabalho apresenta um sistema de controle de estoque para um mercado de bairro, desenvolvido em Java com Spring Boot, interface Thymeleaf e PostgreSQL. O objetivo é demonstrar a avaliação de qualidade de software por meio de requisitos verificáveis, testes funcionais e não funcionais e análise de resultados. O sistema permite cadastrar produtos e categorias, registrar compras, vendas e perdas, consultar histórico e controlar perfis de acesso. A avaliação técnica inclui testes automatizados com banco real, concorrência, integridade transacional, autenticação, cobertura, desempenho e recuperação. A suíte executou 38 casos sem falhas, com 97,81% das linhas e 98,00% dos ramos cobertos na camada de serviços. Esses resultados indicam atendimento aos cenários automatizados, sem comprovar ausência de defeitos. A avaliação de usabilidade com três participantes permanece pendente e deve compor a conclusão acadêmica final.

Palavras-chave: qualidade de software; Java; controle de estoque; testes; PostgreSQL.

Uso deste relatório

A fonte integral está na pasta src, acompanhada de migrations, scripts de operação e evidências. O documento REQUISITOS.pdf define o escopo. As exigências acadêmicas são separadas das decisões técnicas do projeto, e os requisitos específicos do trabalho de DSD não foram transferidos para esta APS.

Sumário

1 Introdução e objetivos 4

2 Fundamentação técnica 4

3 Metodologia 4

4 Desenvolvimento e modelo de dados 5

5 Plano de qualidade 7

6 Resultados técnicos 7

7 Problemas encontrados e melhorias 9

8 Conclusões e recomendações 9

Referências 9

Apêndice A Ficha individual 11

Apêndice B Arquivos da entrega 11

No Word, clique no sumário e escolha Atualizar campo e Atualizar o índice inteiro após completar os dados do grupo.

1 Introdução e objetivos

O controle de estoque de um pequeno mercado depende de registros consistentes de entradas e saídas. O projeto usa esse contexto para exercitar orientação a objetos, banco de dados, engenharia de software e avaliação de qualidade. A aplicação é um protótipo acadêmico de uma loja, com produtos em unidades inteiras e dois perfis de acesso.

O objetivo geral é implementar e avaliar um sistema capaz de manter saldos e histórico coerentes. Os objetivos específicos são definir regras observáveis, estruturar a aplicação em camadas, verificar concorrência e segurança, medir cobertura e latência e documentar limitações e melhorias.

O escopo abrange catálogo, categorias, contas, entradas por compra, saídas por venda ou perda, painel e histórico. Ficam fora caixa financeiro, pagamentos, notas fiscais, clientes, fornecedores, estorno, lotes, validade e ajustes de inventário. A estimativa usa o último custo de compra e não representa lucro contábil.

2 Fundamentação técnica

Testes funcionais comparam o comportamento observado com requisitos de negócio. Testes não funcionais avaliam atributos como desempenho, segurança, recuperação e facilidade de uso. A definição de metas antes da execução permite que a conclusão seja verificável, embora nenhum conjunto finito de testes demonstre ausência de defeitos.

A transação de banco agrupa alterações que devem ocorrer juntas. Neste projeto, atualizar o saldo e inserir histórico formam uma unidade. O bloqueio de linha do PostgreSQL impede que duas saídas concorrentes validem o mesmo saldo anterior. A idempotência evita que um reenvio crie outra movimentação, associando a chave da solicitação ao conteúdo e ao usuário.

A autenticação identifica a conta; a autorização restringe operações ao perfil. Spring Security aplica essas proteções no servidor e inclui proteção CSRF nos formulários. Senhas são armazenadas como hash BCrypt. A interface usa escape HTML e a camada de dados envia parâmetros separadamente do comando SQL.

3 Metodologia

Os requisitos do PDF foram transformados em regras de serviço, restrições de banco e cenários T01 a T25. A implementação mantém Controller, Service, Repository e Model

separados. Os casos automatizados executam serviços reais e requisições MVC com PostgreSQL temporário. As evidências são preservadas por tipo de avaliação.

Componente	Versão ou configuração
Java	JDK 21, distribuição de avaliação Temurin 21.0.13
Aplicação	Spring Boot 3.5.16 e dependências geridas pelo BOM
Construção	Maven 3.9.11 com Wrapper
Banco de avaliação	PostgreSQL 14.19 real, temporário
Interface	Thymeleaf, HTML e CSS locais
Qualidade	JUnit, Spring MockMvc, JaCoCo 0.8.14 e scripts HTTP

O ambiente de execução é compartilhado. Os testes de carga usam loopback e cinco sessões locais. A verificação da instalação no computador da turma e os resultados com participantes devem ser registrados separadamente. A alternativa Docker é distribuída com o código, mas não foi executada neste ambiente de avaliação.

4 Desenvolvimento e modelo de dados

O navegador acessa o servidor Spring. O filtro de segurança verifica a sessão e as permissões. O Controller valida o formulário e prepara a resposta. O Service centraliza as regras e a transação. O Repository usa JdbcTemplate para executar SQL parametrizado. O banco mantém chaves, unicidade, validações e bloqueios.

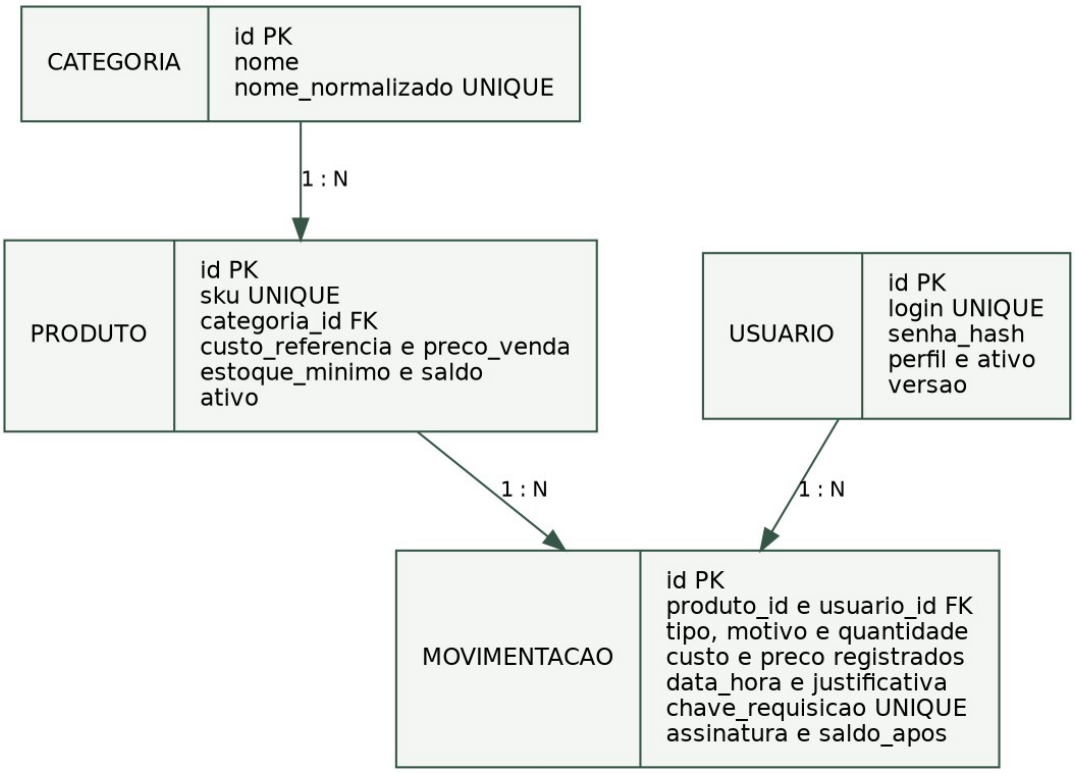


Figura 1 Modelo de relacionamento implementado. Fonte: código do projeto.

Categoria classifica produtos. Produto e usuário se relacionam com movimentações. O histórico guarda quantidade, motivo, custo, preço de venda quando aplicável, data, justificativa, chave, assinatura e saldo após a operação. Os nomes exibidos são os atuais do cadastro; os valores financeiros históricos permanecem fixos.

A confirmação usa um rascunho revisado na sessão. O servidor bloqueia a chave da solicitação, verifica se já houve gravação, bloqueia o produto e valida saldo e preço. Se qualquer etapa falhar, a transação desfaz saldo e histórico. Um trigger impede editar ou excluir movimentações confirmadas.

As rotas administrativas exigem ADMIN. O operador pode consultar e movimentar estoque. Alterações de acesso incrementam a versão da conta e encerram sessões antigas na próxima requisição. Um bloqueio administrativo serializa mudanças que poderiam retirar o último administrador.

A implementação completa acompanha a pasta src, e os scripts SQL estão em src/main/resources/db/migration. A matriz em MATRIZ_REQUISITOS.md liga cada

requisito aos arquivos e testes. A documentação ARQUITETURA.md detalha os mecanismos e apresenta diagramas editáveis em Mermaid.

5 Plano de qualidade

Aspecto	Meta
Correção	100% dos críticos e pelo menos 95% dos funcionais
Integridade	Nenhum saldo negativo ou divergente
Acesso	Bloqueio de todas as tentativas indevidas previstas
Desempenho	p95 de consultas até 2 s e movimentações até 3 s; erros abaixo de 1%
Usabilidade	9 de 9 tarefas sem ajuda e mediana de saída até 60 s
Cobertura	Pelo menos 80% das linhas e 70% dos ramos em service
Recuperação	Contagens, saldos e histórico iguais após restauração

A carga planejada contém mil produtos e dez mil movimentações. O roteiro usa 30 segundos de aquecimento e três rodadas de dois minutos com cinco usuários virtuais. Cada usuário executa ciclos de sete consultas e três movimentações, com pausa de 50 ms. A base inicial é reconstruída entre rodadas. A latência de movimentação inclui formulário, revisão, confirmação e redirecionamento.

A avaliação humana contém três tarefas por participante: localizar produto, registrar entrada de cinco unidades e saída de duas. O formulário USABILIDADE.csv está em branco. O grupo deve registrar tempos, ajuda e dificuldades e comparar o resultado com as metas. Automação no navegador não substitui essa etapa.

6 Resultados técnicos

A execução de verify concluiu 38 testes, sem falhas, erros ou testes ignorados. A suíte inclui os cenários funcionais T01 a T20, proteção CSRF T25 e casos adicionais de fronteira, concorrência e renderização. T21, T23 e T24 possuem evidências próprias. T22 depende de participantes reais.

Métrica	Resultado observado
Casos JUnit	38 aprovados de 38 executados
Linhas em service	134 cobertas, 3 não cobertas, total 137; 97,81%
Ramos em service	98 cobertos, 2 não cobertos, total 100; 98,00%
Exclusões da camada	Nenhuma classe de service excluída do cálculo

Os testes de estoque confirmaram que duas saídas simultâneas de quatro unidades sobre saldo cinco resultam em apenas uma saída aceita. Os reenvios mantêm um registro por chave. Uma falha inserida entre a atualização do saldo e o histórico provoca rollback. A suíte também verificou a proteção do último administrador sob concorrência.

Rod.	Operação	N	Med. ms	p95 ms	Máx. ms	Erros
1	consulta	7204	5.2	19.4	50.2	0
1	movimentacao	3077	9.5	15.2	34.8	0
2	consulta	7153	5.5	20.4	52.0	0
2	movimentacao	3055	9.9	16.7	31.1	0
3	consulta	7100	5.6	21.0	61.1	0
3	movimentacao	3037	9.6	18.5	46.7	0

As medições acima pertencem ao ambiente descrito em evidencias/desempenho/resultado.json. O CSV mantém as amostras individuais. Os resultados não devem ser apresentados como garantia de latência em hospedagem externa.

A restauração em banco separado foi aprovada. A comparação verifica contagens e hashes dos registros, além da conciliação de saldo. A evidência T24 está em evidencias/backup.json; o dump temporário com hashes de contas não integra o pacote público.

A cobertura alta representa execução de linhas e decisões, não um certificado de qualidade. Permanecem limitações: amostra de carga local, ausência de avaliação humana concluída e

ausência de teste de publicação externa. Os resultados reais do grupo devem complementar a análise.

7 Problemas encontrados e melhorias

Na preparação, houve bloqueios do ambiente: compilador Java ausente, confiança TLS do proxy, socket Unix indisponível e tentativa de anexação dinâmica do Mockito. O processo foi ajustado para usar JDK completo, certificados confiáveis, PostgreSQL em TCP local e mock maker por subclasses. A suíte completou após os ajustes. Esses impedimentos não foram apresentados como defeitos de estoque.

As próximas melhorias são aplicar a avaliação de usabilidade e priorizar dificuldades observadas; projetar ajustes auditáveis para eventual uso comercial; separar permissões de migração e execução em implantação externa; e avaliar paginação do agrupamento do painel para catálogos maiores. Cada melhoria deve preservar o histórico e repetir testes afetados.

8 Conclusões e recomendações

A implementação atende aos comportamentos técnicos verificados nos cenários automatizados, com mecanismos explícitos para consistência de estoque, autorização e rastreabilidade. As metas de cobertura foram atingidas. Desempenho e recuperação devem ser interpretados com o ambiente e os dados das evidências, e não generalizados para todas as condições de uso.

A conclusão acadêmica permanece condicionada à avaliação com três participantes, à revisão do relatório e à participação real dos integrantes. O grupo deve preencher seus dados, atualizar o sumário, conferir a normalização institucional, registrar atividades e realizar a apresentação. A experiência reforça que qualidade depende de critérios claros e resultados observáveis, além da implementação de funcionalidades.

Referências

UNIVERSIDADE PAULISTA. Requisitos da APS de Avaliação e Melhoria da Qualidade em Sistemas de Software. Documento 2026_2_CC_SI_8.pdf, segundo semestre de 2026. Consultar o enunciado original da turma.

PROJETO FACULDADE. Requisitos completos da APS Estoque Bairro em Java. 9 nov. 2026. Documento base disponibilizado pelo usuário e incluído neste pacote.

SPRING. System Requirements: Spring Boot 3.5. Disponível em: https://docs.spring.io/spring-boot/3.5/system-requirements.html. Acesso em: 9 nov. 2026.

SPRING. Cross Site Request Forgery: Spring Security. Disponível em: https://docs.spring.io/spring-security/reference/servlet/exploits/csrf.html. Acesso em: 9 nov. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. Explicit Locking. Disponível em: https://www.postgresql.org/docs/current/explicit-locking.html. Acesso em: 9 nov. 2026.

Apêndice A Ficha individual

Preencha uma ficha por integrante, com atividades realmente realizadas. Use a ficha oficial da instituição se o professor exigir. Este modelo não representa comprovação de participação.

Nome: _____

RA: _____

Turma: _____

Data	Atividade realizada	Evidência	Horas

Dificuldades e aprendizado: _____

_____

Assinatura: _____ Data: _____

Apêndice B Arquivos da entrega

Código integral: src e pom.xml. Banco: migrations. Instalação: README.md, Dockerfile, compose.yaml e scripts. Testes: src/test. Resultados: evidencias e RESULTADOS.md. Requisitos: REQUISITOS.pdf e MATRIZ_REQUISITOS.md. Roteiro oral: ROTEIRO_APRESENTACAO.md. Slides: APRESENTACAO_APS.pptx. Avaliação humana: USABILIDADE.csv. Atividades individuais: FICHA_INDIVIDUAL.md.